

Reviewer Pack — Minimal Geometric Entropy Bound for Communication Complexity

Entry 35e2148a9a07 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 20:31:48 UTC

Minimal Geometric Entropy Bound for Communication Complexity

Entry ID: 35e2148a9a07 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 20:31:48 UTC

1. Conjecture

Field A (mathematical branch): Geometric Information Theory **Field B** (complexity object): Communication Complexity

Statement:

For every n -bit communication protocol P computing a boolean function f , the geometric entropy of the distribution of inputs and outputs in P is bounded by $O(n \log n)$, i.e., $H(G(P)) = \Theta(n \log n)$, where H denotes the Shannon entropy and $G(P)$ is the associated metric space representing P .

Rationale (proposer's reasoning):

Geometric information theory provides tools to study the information content of geometric structures. Applying this field to communication complexity could reveal new insights into the intrinsic difficulty of distributing computations among parties, potentially leading to improved complexity lower bounds for certain problems. The use of geometric entropy as a measure links the abstractness of geometry with the quantitative nature of complexity theory.

Taxonomy category: geometric_information_theory_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e2d2572094c7999f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all seeds, the geometric entropy $H(G(P))$ of communication protocols P computing n -bit boolean functions f is within a 95% confidence interval of $\Theta(n \log n)$ across at least 70% of trials, and no seed shows a lack of linear correlation between $H(G(P))$ and $n \log n$. The conjecture is falsified if any seed fails to meet these conditions or shows no linear correlation in at least 30 trials.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric information theory" AND "communication complexity" AND "entropy bound" - "Shannon entropy" in ("communication complexity" OR "geometric information theory") AND metric space - "boolean function" AND communication protocol AND "geometric entropy"

Top relevant hits considered: - [http://arxiv.org/abs/1203.3322v1] A note on Shannon entropy - [http://arxiv.org/abs/quant-ph/0305062v2] Renyi extrapolation of Shannon entropy - [http://arxiv.org/abs/1706.07735v2] Shannon Entropy Reinterpreted - [http://arxiv.org/abs/quant-ph/0607174v1] Exponential Separation of Quantum and Classical One-Way Communication Complexity for a Boolean Function - [http://arxiv.org/abs/quant-ph/0510050v2] Constant communication complexity protocols for multiparty accumulative boolean functions - [http://arxiv.org/abs/2406.07088v1] The Influence of Placement on Transmission in Distributed Computing of Boolean Functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def shannon_entropy(p):
        return -sum(x * math.log2(x) for x in p if x > 0)

    n = 10 # Start with a small instance size and increase as needed
    instances_tested = 0
    total_entropy = 0.0
    n_max = n

    while True:
        f = [random.choice([0, 1]) for _ in range(2**n)]
        protocol = {}

        # Simulate a simple communication protocol (e.g., random assignment)
        for x in range(2**n):
            y = random.choice(f)
```

```

        if y not in protocol:
            protocol[y] = []
        protocol[y].append(x)

    # Compute the geometric entropy
    p = [len(v) / 2**n for v in protocol.values()]
    entropy = shannon_entropy(p)
    total_entropy += entropy
    instances_tested += 1

    if n_max < n:
        n_max = n

    if instances_tested >= 30:
        break

    n += 5 # Increase instance size to test for asymptotic behavior

metric_value = total_entropy / instances_tested
conjecture_holds = False
counterexample = ""

return {
    "metric_name": "geometric entropy",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and sum(1 for r in results if not r["conjecture_holds"]) < 5:
        print(f"RESULT: FALSE counterexample=\"not enough support\" first_failing_seed={seeds[0]}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(seeds)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete the required number of trials and verify the conjecture's conditions. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the allocated time. If the issue persists, consider alternative methods for testing the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19660
2	propose	ollama_remote	glm4:latest	0	0	13170
3	propose	ollama_remote	glm4:latest	0	0	12444
4	preregistration	ollama_remote	glm4:latest	0	0	10380
5	novelty	ollama_remote	glm4:latest	0	0	8354
6	novelty	ollama_remote	glm4:latest	0	0	9375
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16649
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10270
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10273
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7886
11	judge	ollama_remote	glm4:latest	0	0	53321

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 171782 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/35e2148a9a07.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/35e2148a9a07.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/35e2148a9a07.tar.gz* (if generated)